AI Operating System Workshop

Day 1: Data Engineering & RAG Foundation

X-Company
2026

Section 1: RAG Overview

Retrieval-Augmented Generation (RAG) is an AI architecture that combines information retrieval with large language model generation. Instead of relying solely on the model's internal parameters, RAG fetches relevant context from an external knowledge base at query time, then passes it to the LLM for grounded, accurate answers.

Architecture Flow:

- User Query → Embedding → Vector Search → Retrieved Chunks
- Retrieved Chunks + Query → LLM Prompt → Generated Answer

RAG vs Fine-tuning

Criterion	RAG	Fine-tuning
Knowledge update	Real-time (update vector DB)	Requires retraining
Cost	Low (no GPU training)	High (GPU hours)
Accuracy on new data	High	Low (catastrophic forgetting)
Transparency	Citable sources	Black-box weights
Best for	Dynamic, document-heavy QA	Style/format transfer
Latency	Higher (retrieval step)	Lower (single forward pass)

When to Use RAG vs Fine-tuning

- Use RAG: When knowledge changes frequently, when source citation matters, when data is proprietary/confidential
- Use Fine-tuning: When task requires a specific response style, when domain vocabulary is highly specialized
- Use Both: For best results in enterprise settings — fine-tune for format, RAG for facts

Section 2: Document Processing

PDF/DOCX Text Extraction with PyMuPDF

```
import fitz  # PyMuPDF

def extract_pdf(path: str) -> str:
    doc = fitz.open(path)
    text = ''
    for page in doc:
        text += page.get_text('text')
    return text

# DOCX extraction
from docx import Document

def extract_docx(path: str) -> str:
    doc = Document(path)
    return '\n'.join([p.text for p in doc.paragraphs])

text = extract_pdf('document.pdf')
print(text[:500])
```

Thai NLP: Word Segmentation Challenges

Thai language has no spaces between words, making tokenization a critical and non-trivial step. Challenges include:

- No whitespace delimiters — word boundaries must be inferred
- Ambiguous segmentation: 'ตากลม' can mean 'round eyes' or 'Tak Lom (city)'
- New words and loanwords not in dictionary
- Context-dependent meaning shifts

PyThaiNLP Word Tokenization

```
from pythainlp.tokenize import word_tokenize

text = 'การประมวลผลภาษาธรรมชาติมีความสำคัญมาก'

# Default engine: newmm
tokens_newmm = word_tokenize(text, engine='newmm')
print('newmm:', tokens_newmm)

# Longest match
tokens_longest = word_tokenize(text, engine='longest')
print('longest:', tokens_longest)

# attacut (deep learning)
tokens_attacut = word_tokenize(text, engine='attacut')
print('attacut:', tokens_attacut)
```

Thai Tokenizer Comparison

Tokenizer	Algorithm	Speed	Accuracy	Best For
newmm	Dict + MaxMatch	Fast	Good	General purpose
longest	Longest match	Fast	Moderate	Simple texts
attacut	Deep Learning (CNN)	Medium	High	Complex/formal text
icu	Unicode rules	Very Fast	Low	Quick tokenization

Section 3: Chunking Strategies

Fixed-Size Chunking

Split text into equal-length chunks with optional overlap. Simple and fast.

```
def fixed_chunk(text: str, size: int = 512, overlap: int = 50) -> list[str]:
    chunks = []
    start = 0
    while start < len(text):
        end = start + size
        chunks.append(text[start:end])
        start += size - overlap
    return chunks

chunks = fixed_chunk(text, size=512, overlap=50)
print(f'Total chunks: {len(chunks)}')
```

Recursive Chunking

Splits on natural boundaries (paragraphs → sentences → words) until target size is reached.

```
from langchain.text_splitter import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=512,
    chunk_overlap=50,
    separators=['\n\n', '\n', '.', ' ', '']
)
chunks = splitter.split_text(text)
print(f'Chunks: {len(chunks)}, Sample: {chunks[0][:100]}')
```

Semantic Chunking

Uses embedding similarity to split at meaningful topic boundaries.

```
from langchain_experimental.text_splitter import SemanticChunker
from langchain_community.embeddings import OllamaEmbeddings

embeddings = OllamaEmbeddings(model='bge-m3')
chunker = SemanticChunker(
    embeddings,
    breakpoint_threshold_type='percentile',
    breakpoint_threshold_amount=95
)
chunks = chunker.split_text(text)
```

Chunking Strategy Comparison

Strategy	Pros	Cons	Best Use Case
Fixed-size	Simple, fast, predictable	May split mid-sentence	Large uniform documents
Recursive	Respects natural boundaries	Slightly slower	General-purpose RAG
Semantic	Coherent topic chunks	Slow, needs embeddings	Research papers, reports

Section 4: Embeddings & Vector Database

Embeddings are dense vector representations of text that capture semantic meaning. Similar texts produce similar vectors, enabling semantic search.

Embedding Model Comparison

Model	Dimensions	Languages	Size	Best For
bge-m3	1024	100+	570MB	Multilingual, Thai-friendly
nomic-embed-text	768	English primary	274MB	English docs, fast
multilingual-e5-large	1024	100+	560MB	Cross-lingual retrieval

Qdrant Vector DB Setup

```
# Start Qdrant with Docker
docker run -d --name qdrant \
```

```
  -p 6333:6333 -p 6334:6334 \
  -v $(pwd)/qdrant_storage:/qdrant/storage \
  qdrant/qdrant:latest
```

Indexing Documents

```
from qdrant_client import QdrantClient
from qdrant_client.models import Distance, VectorParams, PointStruct
import ollama

client = QdrantClient('localhost', port=6333)

# Create collection
client.create_collection(
    collection_name='workshop_docs',
    vectors_config=VectorParams(size=1024, distance=Distance.COSINE)
)

# Index chunks
for i, chunk in enumerate(chunks):
    embedding = ollama.embeddings(model='bge-m3', prompt=chunk)
    client.upsert('workshop_docs', points=[
        PointStruct(id=i, vector=embedding['embedding'],
                    payload={'text': chunk})
    ])
```

Semantic Search

```
def search(query: str, top_k: int = 5) -> list[str]:
    query_vec = ollama.embeddings(model='bge-m3', prompt=query)['embedding']
    results = client.search(
        collection_name='workshop_docs',
        query_vector=query_vec,
        limit=top_k
    )
    return [r.payload['text'] for r in results]

context = search('What is the refund policy?')
```

Section 5: Hybrid Search

Hybrid search combines dense vector similarity (semantic) with sparse keyword matching (BM25) for better retrieval coverage.

Dense vs BM25

Approach	Method	Strength	Weakness
Dense (Vector)	Embedding similarity	Semantic understanding	Misses exact keywords
BM25 (Sparse)	TF-IDF based	Exact keyword matching	No semantic understanding
Hybrid	Weighted combination	Best of both	More complex pipeline

Hybrid Search Formula

```
# Reciprocal Rank Fusion (RRF)
def rrf_score(rank: int, k: int = 60) -> float:
    return 1.0 / (k + rank)
```

```
# Combine dense + sparse results
def hybrid_search(query, dense_results, sparse_results, alpha=0.7):
    scores = {}
    for rank, doc in enumerate(dense_results):
        scores[doc.id] = alpha * rrf_score(rank)
    for rank, doc in enumerate(sparse_results):
        scores[doc.id] = scores.get(doc.id, 0) + (1-alpha) * rrf_score(rank)
    return sorted(scores.items(), key=lambda x: x[1], reverse=True)
```

Reranking Strategies

- Cross-encoder reranking: Use a fine-tuned model (e.g., bge-reranker-v2-m3) to score query-document pairs
- LLM-as-reranker: Ask LLM to rank retrieved passages by relevance
- Cohere Rerank API: Commercial API for production reranking

Section 6: Basic RAG Pipeline

```
import ollama
from qdrant_client import QdrantClient

client = QdrantClient('localhost', port=6333)

def rag_pipeline(question: str) -> str:
    # 1. Embed the question
    q_vec = ollama.embeddings(model='bge-m3', prompt=question)['embedding']

    # 2. Retrieve top-5 relevant chunks
    results = client.search(
        collection_name='workshop_docs',
        query_vector=q_vec, limit=5
    )
    context = '\n\n'.join([r.payload['text'] for r in results])

    # 3. Build prompt
    prompt = f'Context:\n{context}\n\nQuestion: {question}\nAnswer:'

    # 4. Generate answer
    response = ollama.chat(
        model='llama3.2',
        messages=[{'role': 'user', 'content': prompt}]
    )
    return response['message']['content']

answer = rag_pipeline('What documents are required for reimbursement?')
print(answer)
```

Expected Output: A concise, grounded answer based only on retrieved context. The model will cite relevant sections rather than hallucinating.

Section 7: Hands-On Exercises

Exercise 1: Document Ingestion Pipeline

Task: Build a pipeline to extract text from 5 PDFs and store chunks in Qdrant.

- Hint: Use PyMuPDF for extraction, RecursiveCharacterTextSplitter for chunking
- Bonus: Add metadata (filename, page number) to each chunk

Exercise 2: Thai Text Processing

Task: Compare tokenization results of newmm vs attacut on a Thai product description.

- Hint: Use pythainlp.tokenize.word_tokenize with engine parameter

Exercise 3: Hybrid Search

Task: Implement RRF to combine dense and BM25 results, compare with dense-only.

- Hint: Use rank_bm25 library for sparse retrieval

Exercise 4: Reranking

Task: Add a cross-encoder reranker to your RAG pipeline and measure NDCG improvement.

- Hint: sentence-transformers CrossEncoder with 'BAAI/bge-reranker-v2-m3'

Exercise 5: End-to-End RAG

Task: Build a complete Q&A chatbot over a company policy document with conversation history.

- Hint: Use a list to maintain chat_history, prepend context to each turn

Section 8: Reference & Resources

Libraries

- PyMuPDF: https://pymupdf.readthedocs.io
- PyThaiNLP: https://pythainlp.github.io
- Qdrant: https://qdrant.tech/documentation
- LangChain Text Splitters: https://python.langchain.com/docs/modules/data_connection/document_transformers
- Ollama: https://ollama.ai

Recommended Reading

- "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks" — Lewis et al. (2020)
- "BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation" — Thakur et al.
- Qdrant Hybrid Search Documentation
- BGE-M3 Technical Report (BAAI)